JAIN (Deemed-to-be University)

Department of Computer Science and IT

ACTIVITY 2

Student Research Report

NLP-Based Intelligent Writing Assistant: Autocorrect, Autocomplete, and POS-Aware Suggestions

Team Name: SyntaxSquad

Submitted by

Student 1: Anoon M R (25MCAR0179)
Student 2: Aman Kumawat (25MCAR0023)
Student 3: Adithya Krishnan T N (25MCAR0003)

Course: Natural Language Processing (25MCAAI303)
Faculty: Dr. Sharal Coelho
Academic Year: 2026–2027

Submission Date: _____

Contents

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 2

2 Literature Survey **3**

2.1 Comparison of Existing Research 3

2.2 Research Gap 5

3 Methodology **6**

3.1 Proposed Approach 6

3.2 Dataset Description 6

3.3 Algorithms and Technologies 7

3.4 POS-Aware Suggestion Filtering 7

3.5 System Workflow 8

4 Experimental Details and Results **9**

4.1 Experimental Environment 9

4.2 Evaluation Metrics 9

4.3 Evaluation Set Generation 10

4.4 Experimental Results 10

4.5 Confusion Matrix 11

4.6 Output Screenshots 12

5 Error Analysis **13**

5.1 Observed Errors 13

5.2 Possible Causes 14

5.3 Improvement Strategies 14

6 Project Proof and Implementation Evidence 16

6.1 GitHub Repository 16

6.2 Deployment Evidence 16

6.3 Deployed Application Screenshot 17

7 Conclusion and Future Work 18

7.1 Conclusion 18

7.2 Future Work 18

8 Acknowledgement of AI Usage 19

8.1 AI Usage Statement 19

Chapter 1

Introduction

1.1 Background

Writing assistance tools are among the most widely used applications of Natural Language Processing, embedded in everyday software from mobile keyboards to office suites and browsers. Most commercial tools, however, operate as opaque black boxes built on proprietary, pre-trained models. This project takes the opposite approach: every component – spelling correction, next-word prediction, and part-of-speech-aware filtering – is implemented from first principles in Python, directly applying the statistical NLP techniques covered in Units 1 and 2 of this course (text processing, edit distance, the noisy channel model, N-gram language models, and Hidden Markov Model POS tagging with Viterbi decoding).

1.2 Problem Statement

Everyday digital writing is affected by three recurring issues that basic tools handle poorly:

- **Spelling errors** (typographical mistakes – insertions, deletions, substitutions, and transpositions) are often corrected without considering surrounding context or realistic error likelihoods.
- **Autocomplete suggestions** are frequently generated without regard to grammatical role, leading to suggestions that are statistically common but grammatically implausible in context (e.g. suggesting a noun where a verb is expected).
- **No lightweight, transparent, from-scratch NLP pipeline** was available that demonstrates how edit distance, the noisy channel model, N-gram language modeling, and HMM-based POS tagging work together as a single coherent system.

This project directly addresses all three by building each component from first principles, without relying on pre-trained NLP libraries such as NLTK's built-in taggers or spaCy models.

1.3 Objectives

1. Implement a rule-based tokenizer for sentence and word segmentation.
2. Implement a spelling corrector using the Noisy Channel Model, with Edit Distance candidate generation (up to two edits) and a channel model learned from a confusion matrix of real human misspellings.
3. Implement bigram and trigram N-Gram Language Models with Laplace smoothing (trigram with back-off to bigram) for next-word prediction.
4. Implement a bigram Hidden Markov Model POS tagger with Viterbi decoding, and use it to filter next-word suggestions by expected part of speech.
5. Integrate all four components behind a live Flask web interface, and evaluate each component quantitatively on held-out data.

1.4 Scope of the Project

The system operates entirely offline once the datasets are downloaded, targets English text, and is trained on the Brown Corpus (via NLTK) plus a corpus of real correct-word $\rightarrow$ misspelling pairs used to learn the spelling channel model. The implementation covers the full scope planned in the Phase 1 proposal:

- spelling candidates at edit distance 1 *and* 2;
- a channel model learned from deletion, insertion, substitution and transposition confusion matrices (Kernighan et al., 1990), not hand-chosen weights;
- bigram *and* trigram language models, the trigram backing off to the bigram for unseen contexts;
- a live web interface that re-analyses the text as the user types, without reloading the page.

Remaining limitations (Laplace smoothing, a bigram HMM, and isolated-word spelling correction) are discussed in Chapter 5 (Error Analysis).

Chapter 2

Literature Survey

2.1 Comparison of Existing Research

Table 2.1: Comparative Analysis of Research Papers

Paper Title	**Publication**	**Year**	**Methodology**	**Results / Findings**
A Technique for Computer Detection and Correction of Spelling Errors	Communications of the ACM, 7(3)	1964	Assumes a misspelled word has exactly one error (insertion, deletion, substitution, or transposition) and searches a dictionary for the closest match.	Found that ∼80% of real-world misspellings are single-error misspellings; laid the groundwork for edit-distance-based correction (Damerau–Levenshtein distance).
A Spelling Correction Program Based on a Noisy Channel Model	COLING, Vol. 2, pp. 205–210	1990	Models a typo as the true word passed through a noisy channel; ranks candidates by $P(\text{word}) \times P(\text{typo} \mid \text{word})$, trained on AP newswire text.	Established the noisy-channel formulation this project's spell corrector is directly based on; channel probabilities learned from real confusion data outperform frequency-only ranking.

Paper Title	**Publication**	**Year**	**Methodology**	**Results / Findings**
Estimation of Probabilities from Sparse Data for the Language Model Component of a Speech Recognizer	IEEE Trans. Acoustics, Speech, and Signal Processing, ASP-35(3)	1987	Proposes Katz back-off smoothing: unseen n-grams fall back to lower-order n-gram estimates discounted via Good–Turing.	More accurate than simple Laplace (add-one) smoothing on sparse n-gram counts, directly motivating this project's planned Future Work (Chapter 7).
Class-Based n-gram Models of Natural Language	Computational Linguistics, 18(4), pp. 467–480	1992	Clusters words into classes from co-occurrence statistics and builds n-gram models over classes rather than individual words.	Reduces data sparsity for n-gram prediction; relevant to scaling this project's word-level bigram/trigram LM, whose trigram counts are very sparse.
An Empirical Study of Smoothing Techniques for Language Modeling	Computer Speech & Language, 13(4), pp. 359–394	1999	Empirically compares Jelinek-Mercer, Katz, Witten-Bell, and Kneser-Ney smoothing across corpus sizes and n-gram orders.	Kneser-Ney (and a novel modified variant) consistently outperforms other smoothing methods; directly explains why this project's Laplace-smoothed models have high perplexity, and why the trigram's is even higher than the bigram's (Chapter 5).
A Stochastic Parts Program and Noun Phrase Parser for Unrestricted Text	2nd Conf. on Applied NLP (ANLP), pp. 136–143	1988	Uses dynamic programming (an HMM-style tagger) to balance lexical and contextual tag probabilities, trained on the Brown Corpus.	Reports 95–99% tagging accuracy; this project's own HMM tagger (trained on the same Brown Corpus) is directly comparable in design.

Paper Title	Publication	Year	Methodology	Results / Findings
Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm	IEEE Trans. Information Theory, 13(2), pp. 260–269	1967	Introduces the Viterbi algorithm: a dynamic-programming method for finding the maximum-likelihood path through a trellis of states.	The algorithm this project's POS tagger uses directly for exact most-likely-tag-sequence decoding, originally developed outside NLP for decoding convolutional codes.
TnT – A Statistical Part-of-Speech Tagger	6th Conf. on Applied NLP (ANLP-2000), pp. 224–231	2000	A trigram HMM tagger with careful smoothing and an explicit unknown-word-handling model (suffix-based probability estimation).	Shows a Markov-model (HMM) tagger can match or beat more complex approaches such as Maximum Entropy; its unknown-word handling is a direct source of inspiration for improving this project's current smoothing (Chapter 5).

2.2 Research Gap

While the noisy channel model, n-gram smoothing, and HMM POS tagging are each individually well studied (as shown above), the surveyed literature largely treats them as separate problems solved in isolation, each evaluated on its own benchmark. Few openly documented systems combine all three into a single pipeline where POS tagging output directly filters language-model suggestions in real time for an end user. This project targets that integration gap: using the HMM tagger's transition probabilities to filter the N-gram LM's next-word candidates (Chapter 3, Section 3.4), rather than running the two models independently.

Chapter 3

Methodology

3.1 Proposed Approach

The system is implemented as a five-stage pipeline (Figure 3.1): raw text is tokenized, spell-corrected, tagged for part of speech using an HMM with Viterbi decoding, and the N-gram language model's next-word candidates are then filtered by the POS tag the HMM expects to follow, before being served to the user through a Flask web interface that updates live as the user types.

3.2 Dataset Description

- **Dataset name and source:** Brown Corpus, downloaded from the NLTK data repository (`https://www.nltk.org/nltk_data/`) using NLTK's downloader, together with the Universal tagset mapping: `nltk.download('brown')` and `nltk.download('universal_tagset')`. It is read through `nltk.corpus.brown` and was cleaned in Activity 1.
- **Number of samples:** 54,286 cleaned sentences (from 57,340 raw sentences) for the language model; 54,270 POS-tagged sentences (Universal tagset) for the HMM tagger.
- **Vocabulary:** 24,891 words with frequency ≥ 2 (40,353 unique tokens total before filtering).
- **POS tag set:** 10 Universal tags – ADJ, ADP, ADV, CONJ, DET, NOUN, NUM, PRON, PRT, VERB.
- **Misspelling corpus (channel model):** `spell-errors.txt` from Peter Norvig's *Natural Language Corpus Data* [8], downloaded from `https://norvig.com/ngrams/spell-errors.txt`: 7,841 correct words, each listed with the ways real people misspelled it, giving 38,218 (correct word, misspelling) pairs in total. This corpus is used to learn the spell corrector's confusion matrix and to test it on genuine human errors.

- **Preprocessing steps (Activity 1):** lowercasing, punctuation stripping, regex tokenization, filtering tokens shorter than 2 characters, and discarding sentences with fewer than 3 tokens.

3.3 Algorithms and Technologies

- **Language:** Python 3.12
- **Web framework:** Flask 3.1.3
- **Tokenizer:** Regex-based sentence and word segmentation, preserving contractions (e.g. `isn't`).
- **Spell Corrector:** Noisy Channel Model [2]. For a misspelled word x, candidate corrections c are all vocabulary words within edit distance 1 or 2 of x (insertion, deletion, substitution, transposition), ranked by

$$\hat{c} = \arg\max_{c} \; P(c)\, P(x \mid c),$$

where $P(c)$ is the Laplace-smoothed unigram prior from the Brown Corpus vocabulary and $P(x \mid c)$ is the channel probability. The channel model is learned: every single-edit (correct, misspelling) pair in the training split of the misspelling corpus is aligned to its one edit, which is counted in one of four confusion matrices (`del`, `ins`, `sub`, `trans`), keyed on the characters involved. For example,

$$P(\text{del } y \text{ after } x) = \frac{\text{del}[x, y] + 1}{\sum_{y'} \text{del}[x, y'] + 26}.$$

12,356 single-edit training pairs fill the matrices. For an edit-distance-2 candidate, the two edits' probabilities are multiplied.

- **N-Gram Language Model:** Bigram and trigram models, both with Laplace (add-one) smoothing:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}, \qquad P(w_i \mid w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i) + 1}{C(w_{i-2}, w_{i-1}) + V},$$

where V is vocabulary size. If the two-word context (w_{i-2}, w_{i-1}) never occurred in training, the trigram backs off to the bigram estimate. Next-word prediction uses the trigram whenever two previous words are available.

- **HMM POS Tagger:** Bigram HMM with Laplace-smoothed transition (A), emission (B), and initial (π) probabilities, decoded with the Viterbi algorithm for exact most-likely tag sequences.

3.4 POS-Aware Suggestion Filtering

After the HMM tags the corrected input, the tagger's transition matrix is queried for $\arg\max_t P(t \mid \text{last_tag})$ – the POS tag most likely to follow the current context. Each

candidate word from the N-gram model is independently assigned its own most-likely tag via $P(\text{tag} \mid \text{word}) \propto P(\text{word} \mid \text{tag})\, P(\text{tag})$. From the N-gram model's top 15 candidates, only those whose tag matches the expected tag are kept, and the top 5 are shown. If no candidate matches, the unfiltered list is shown instead, so the user always gets suggestions. This is what distinguishes the system's suggestions from a plain frequency-ranked autocomplete.

3.5 System Workflow

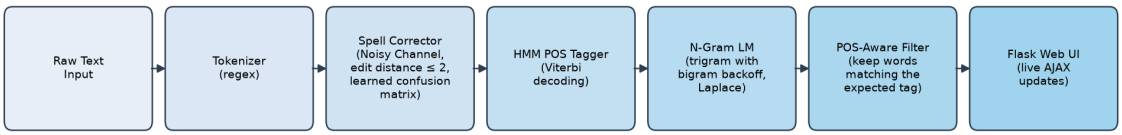


Figure 3.1: Proposed System Workflow

Chapter 4

Experimental Details and Results

4.1 Experimental Environment

- **Programming language:** Python 3.12
- **Development environment:** Claude Code CLI + local Python interpreter, Windows 11
- **Libraries and frameworks:** Flask 3.1.3, NLTK 3.8.1 (Activity 1 only), matplotlib 3.11.1 (evaluation plots)
- **Hardware configuration:** Standard laptop CPU required – all models are simple frequency/probability tables, not neural networks
- **Dataset split:** 90% train / 10% test, fixed random seed (42), applied independently to three datasets: the sentence set (48,858 train / 5,428 test, for the LM), the POS-tagged sentence set (for the HMM tagger), and the misspelling corpus (34,397 train / 3,821 test pairs, for the spell corrector's confusion matrix). Every model is trained on its train split only and scored on its test split.

4.2 Evaluation Metrics

Five metrics were used across the three models:

- **Spell Corrector (synthetic typos):** Top-1 correction accuracy on an auto-generated evaluation set of 200 sentences, each with exactly one induced edit-distance-1 typo (see Section 4.3).
- **Spell Corrector (real-world misspellings):** Top-1 correction accuracy on 500 held-out (correct, misspelling) pairs from the misspelling corpus – genuine human errors that the confusion matrix never saw during training.
- **N-Gram LM:** Bigram and trigram perplexity on the held-out 10% test sentences – $\mathrm{PP} = \exp\left(-\frac{1}{N}\sum \ln P(w_i \mid \text{context})\right)$.
- **HMM POS Tagger:** Per-word tagging accuracy against gold Universal-tagset labels on 500 held-out test sentences.

4.3 Evaluation Set Generation

Rather than hand-curating 200 test sentences, the evaluation set was generated programmatically from held-out (test-split) Brown Corpus sentences: for each sentence, one word was randomly selected and replaced with a random edit-distance-1 variant (an insertion, deletion, substitution, or transposition), simulating a realistic single-character typo. This guarantees the "correct answer" for each typo is known exactly, allowing automated accuracy scoring. The generated set is saved as `report/evaluation_set.json`.

4.4 Experimental Results

Table 4.1: Performance of the Proposed System

Evaluation Metric	**Obtained Value**
Spell Corrector top-1 accuracy (synthetic typos)	76.00% (152 / 200)
Spell Corrector top-1 accuracy (real-world misspellings)	38.00% (190 / 500)
N-Gram LM bigram perplexity (held-out)	5595.97
N-Gram LM trigram perplexity (held-out, with back-off)	13641.36
POS Tagger word accuracy (held-out)	91.74% (8242 / 8984)

All values are produced by `src/evaluate.py`, which saves them to the file `report/evaluation_results.json`. Re-running the script reproduces them exactly, because the random seed is fixed.

Spell corrector. On synthetic single-edit typos the corrector recovers the original word 76% of the time. On genuine human misspellings it scores 38%, but this number has to be read alongside how hard each misspelling is. Table 4.2 breaks the 500 real test pairs down by their edit distance from the correct word.

Table 4.2: Real-world spelling accuracy by edit distance

Edit distance (correct → misspelling)	**Pairs**	**Corrected**
1	173	112 (64.7%)
2	134	78 (58.2%)
3 or more	193	0 (out of reach by design)
Total	**500**	**190 (38.0%)**

193 of the 500 real misspellings (38.6%) are three or more edits away from the intended word, which a corrector that searches up to edit distance 2 cannot reach. On the 307 misspellings that are within reach, the corrector is right 61.9% of the time (190 / 307).

N-gram LM. Both perplexities are high, and the trigram's is higher than the bigram's. Both effects come from Laplace smoothing, not from a counting error; Section 5.2 explains why.

POS tagger. 91.74% word-level accuracy with only Laplace smoothing and no unknown-word model is a strong result for a from-scratch bigram HMM trained on ∼49K sentences.

4.5 Confusion Matrix

The POS tagger is inherently a 10-class classification problem (one class per Universal POS tag), so a confusion matrix was computed on the held-out test set.

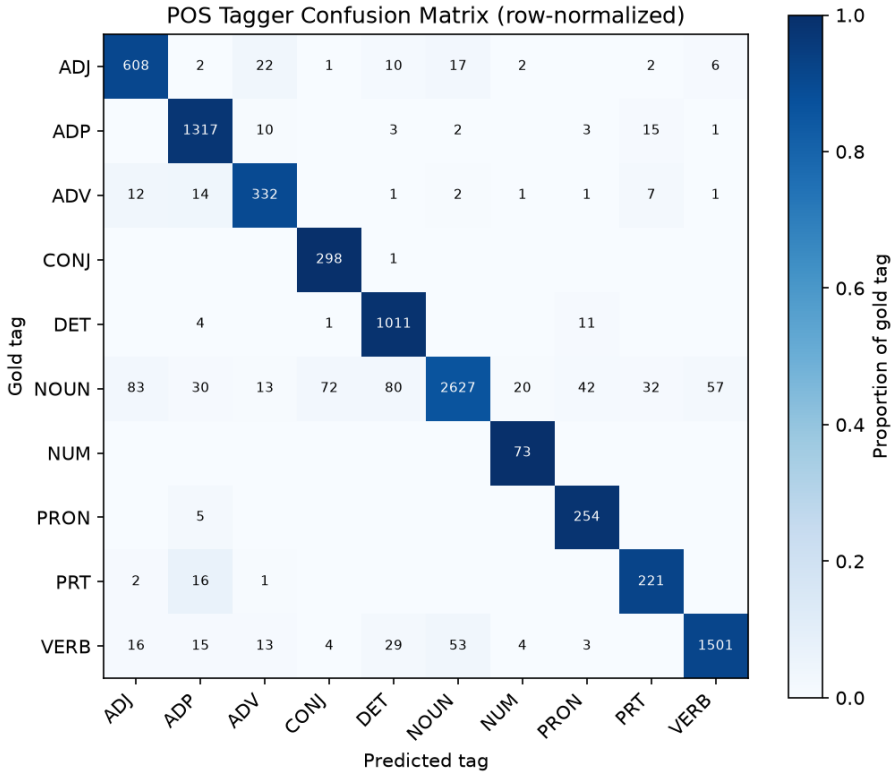


Figure 4.1: Confusion Matrix of the HMM POS Tagger (held-out test set, counts shown, color = row-normalized proportion)

The matrix is strongly diagonal (correct predictions dominate every row). The largest off-diagonal confusions are: NOUN misclassified as CONJ (72 instances) and ADJ (83 instances); and VERB/NOUN mutual confusion (53–57 instances each way). These are discussed in Chapter 5.

4.6 Output Screenshots

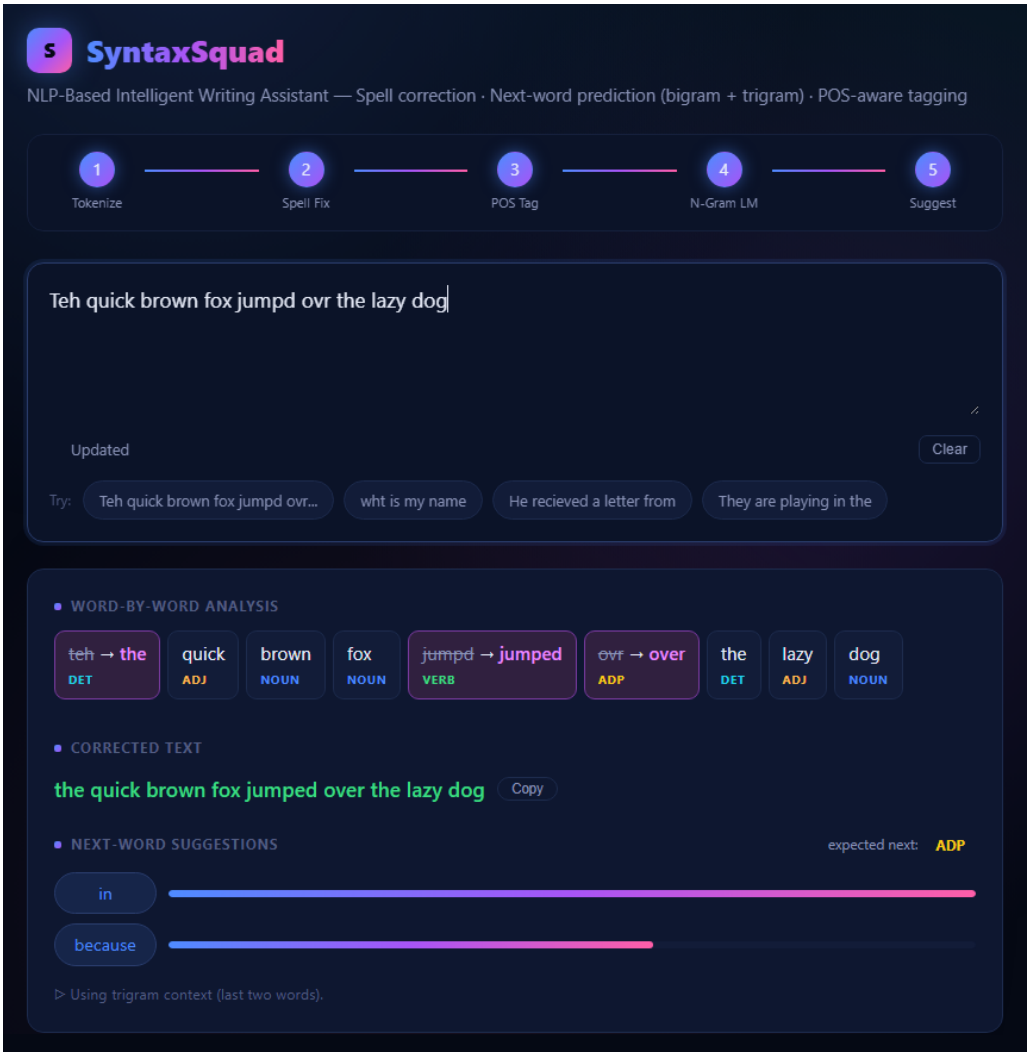


Figure 4.2: Live output of the Flask web app for the input “Teh quick brown fox jumpd ovr the lazy dog”. All three typos are fixed (teh → the, jumpd → jumped, ovr → over; the strikethrough shows the original), every word is POS-tagged, and the next-word suggestions (“in”, “because”) are filtered to the expected ADP tag using trigram context.

Chapter 5

Error Analysis

5.1 Observed Errors

Spell Corrector – why the 48 synthetic misses happened. Each of the 48 synthetic typos the corrector got wrong was inspected and sorted into one of four groups (Table 5.1).

Table 5.1: Breakdown of the 48 synthetic-typo misses

Cause	**Count**	**Example (intended → typo → output)**
A very frequent word two edits away beat the correct word one edit away	23	head → haed → *and*; source → sourhe → *course*
A different one-edit word with a higher score won	11	here → herp → *help*
The typo is itself a real word, so it is never flagged	9	will → gill; the → she
The intended word is not in the vocabulary	5	silkworms → xsilkworms

The largest group (23 of 48) is a side effect of extending the search to edit distance 2: the unigram prior of a very common word such as "and" is large enough to outweigh the extra edit's channel penalty. This is also the main reason synthetic-typo accuracy fell from 85.5% in the earlier, edit-distance-1-only version of the system to 76% now. The trade is deliberate: edit distance 2 is what makes 78 of the real-world misspellings in Table 4.2 correctable at all, and the learned confusion matrix fixed specific failures of the earlier version, such as "ovr" → "or" (now "over") and "wht" → "who" (now "what").

Spell Corrector – out-of-reach real misspellings. 193 of the 500 real-world test misspellings are three or more edits from the intended word (Table 4.2); some are not typos at all but phonetic or alternate spellings. None of these can be corrected by an edit-distance-2 search.

POS Tagger – NOUN confusions. The largest off-diagonal cells in Figure 4.1 are NOUN predicted as ADJ (83), DET (80) and CONJ (72), plus mutual NOUN/VERB

confusion (53–57 each way). These are concentrated on words that can be either a noun or an adjective (e.g. “green”, “public”) or a noun or a verb (“cut”, “show”), and on proper names never seen in training (124 of the 429 NOUN errors), where the bigram HMM’s one-tag context window cannot fully disambiguate. Ambiguous function words such as “that” cause a separate, smaller set of DET/PRON/ADP errors.

N-Gram LM – high perplexity, and trigram higher than bigram. Bigram perplexity is 5,596 and trigram perplexity is 13,641 on held-out data.

5.2 Possible Causes

- The spell corrector scores candidates at edit distance 1 and 2 on equal terms, apart from the channel probability. It has no explicit preference for fewer edits, so a high enough prior lets a two-edit word win.
- The corrector looks at each word in isolation, so a typo that happens to form another real word (“gill” for “will”) is never flagged. Catching these needs the sentence context (a real-word error model).
- The confusion matrix is sparse: 12,356 single-edit training pairs spread over roughly 2,700 possible (character, edit) combinations, so many specific edits have only a handful of counts.
- The HMM tagger’s bigram context (only the immediately preceding tag) is a known limitation relative to trigram HMMs such as TnT (Brants, 2000, surveyed in Chapter 2).
- Laplace smoothing adds the full vocabulary size $V \approx 40{,}000$ to every denominator, regardless of how often the context occurred (Chen & Goodman, 1999). Trigram contexts are far rarer than bigram contexts, so for a typical trigram context $C(w_{i-2}, w_{i-1})$ is tiny compared with V, and almost all the probability mass is spread uniformly over unseen words. That is why the trigram’s perplexity is higher than the bigram’s, even though a trigram has more context: the problem is the smoothing, not the extra context.

5.3 Improvement Strategies

Implemented in the current system:

- A learned confusion matrix replaced the earlier fixed per-edit-type weights. This fixed the “ovr” → “or” and “wht” → “who” errors found during manual testing.
- Spelling candidates were extended to edit distance 2, making 78 real-world misspellings correctable that were previously out of reach.
- The trigram model backs off to the bigram for unseen contexts, so suggestions never collapse to a uniform guess.

- Single-letter words ("I", "a"), which Activity 1's preprocessing excluded from the vocabulary, are special-cased as known words after "I" was being "corrected" to "in" during testing.
- The POS-aware suggestion filter (Section 3.4) was added after testing showed that plain frequency-ranked suggestions ignored grammatical role.

Suggested for future work (not yet implemented):

- Prefer edit-distance-1 candidates whenever one exists (or apply an explicit penalty per extra edit). This directly targets the largest group of synthetic misses.
- Use the language model to rank spelling candidates in context, which would also catch real-word errors such as "gill" for "will".
- Replace Laplace smoothing with Katz back-off or Kneser-Ney smoothing to reduce perplexity, especially for the trigram.
- Extend the HMM to a trigram model for richer context, following the TnT tagger's design.

Chapter 6

Project Proof and Implementation Evidence

6.1 GitHub Repository

Repository URL: https://github.com/AnoonMR/Writing-Assistant

The repository contains the complete Activity 2 project: the source code for all five modules, the evaluation script, the preprocessed datasets, the README with setup instructions, and this report with its presentation. The Activity 1 dataset download and preprocessing scripts are in a separate repository, https://github.com/AnoonMR/nlp-a1.

6.2 Deployment Evidence

Deployment: Not deployed to a public hosting service. The application is designed to run locally: all datasets are downloaded once (Activity 1), after which the entire system – language model, spell corrector, POS tagger, and Flask server – runs fully offline on `localhost`. This was a deliberate scope decision for a course mini-project rather than a limitation of the architecture; nothing in the design prevents deploying it to a hosting service such as Render or PythonAnywhere if required.

To run locally:

1. `pip install -r requirements.txt`
2. `cd src`, then `python app.py`
3. Open http://127.0.0.1:5000 in a browser and start typing; results update live, with no button to press.

To reproduce every number in Chapter 4, run `python evaluate.py` from the same `src` folder.

6.3 Deployed Application Screenshot

See Figure 4.2 (Chapter 4), which shows the live local application running and producing real output.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project implemented a complete, from-scratch NLP pipeline for an intelligent writing assistant, covering all five planned modules: tokenizer, Noisy Channel spell corrector with a learned confusion matrix, bigram and trigram N-gram language models, HMM POS tagger with Viterbi decoding, and a live Flask web interface that ties them together with POS-aware suggestion filtering. All objectives stated in Chapter 1 were achieved, at the full scope planned in the Phase 1 proposal.

Quantitatively, on held-out data the system reaches 91.74% POS-tagging accuracy and 76% top-1 spelling accuracy on synthetic typos. On genuine human misspellings it reaches 38% overall, rising to 61.9% on the misspellings within its edit-distance-2 search range. These are models trained entirely from first principles (no pre-trained taggers or embeddings) on a single mid-sized corpus. Every weak point found – the frequency bias of edit-distance-2 candidates, out-of-range misspellings, and the high perplexity caused by Laplace smoothing – has a measured, explained cause (Chapter 5) and a concrete fix.

7.2 Future Work

- Prefer edit-distance-1 spelling candidates when one exists, or penalise each extra edit, to remove the largest group of spelling errors.
- Rank spelling candidates using the language model in context, which would also catch real-word errors.
- Replace Laplace smoothing with Katz back-off or Kneser-Ney smoothing, which should lower perplexity, especially for the trigram.
- Extend the HMM tagger to trigram context.
- Optionally deploy the Flask app to a public hosting service.

Chapter 8

Acknowledgement of AI Usage

8.1 AI Usage Statement

This project was developed with the assistance of Claude (Anthropic), an AI tool, which was used for code implementation and debugging.

Bibliography

[1] F. J. Damerau, "A technique for computer detection and correction of spelling errors," *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964. DOI: 10.1145/363958.363994.

[2] M. D. Kernighan, K. W. Church, and W. A. Gale, "A spelling correction program based on a noisy channel model," *Proceedings of COLING*, vol. 2, pp. 205–210, 1990.

[3] S. M. Katz, "Estimation of probabilities from sparse data for the language model component of a speech recognizer," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. ASP-35, no. 3, pp. 400–401, 1987.

[4] P. F. Brown, V. J. Della Pietra, P. V. deSouza, J. C. Lai, and R. L. Mercer, "Class-based n-gram models of natural language," *Computational Linguistics*, vol. 18, no. 4, pp. 467–480, 1992.

[5] S. F. Chen and J. Goodman, "An empirical study of smoothing techniques for language modeling," *Computer Speech & Language*, vol. 13, no. 4, pp. 359–394, 1999.

[6] K. W. Church, "A stochastic parts program and noun phrase parser for unrestricted text," *Proceedings of the 2nd Conference on Applied Natural Language Processing (ANLP)*, pp. 136–143, 1988. DOI: 10.3115/974235.974260.

[7] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[8] P. Norvig, "Natural language corpus data," in *Beautiful Data*, T. Segaran and J. Hammerbacher, Eds. O'Reilly Media, 2009, pp. 219–242. Data: https://norvig.com/ngrams/.

[9] T. Brants, "TnT – A statistical part-of-speech tagger," *Proceedings of the 6th Conference on Applied Natural Language Processing (ANLP)*, pp. 224–231, 2000. DOI: 10.3115/974147.974178.